

CASS-Greedy-Quota: A Constructive Theory and Implementation Plan for Cost-Effective Small-Model Committees

Prepared as a self-contained technical report

May 2026

Abstract

Practitioners increasingly want to deploy combinations of smaller and cheaper models for specialized tasks. The hard part is not imagining possible combinations: there are too many. A system designer can vary the subset of models, the context each model sees, the voting or routing rule, the cost budget, and the evaluation protocol. If these choices are searched naively on a small benchmark, the empirical winner can be a validation-set accident. This report develops a constructive alternative. We model each model-context pair as an atom, use greedy residual coverage to select complementary atoms, reduce aggregation search to quota rules, optionally compress large weighted committees into small deployable committees, and use adaptive racing plus final holdout certification to allocate benchmark effort efficiently. The main goal is not to prove that one universal ensemble rule is optimal. The goal is to give practitioners a mathematically justified way to avoid an exponential search space while still building useful multi-model stacks.

Contents

1	Problem statement	4
1.1	What this report is not claiming	4
2	Basic setup	5
2.1	Episodes, cells, and labels	5
2.2	Model-context atoms	5
2.3	Serial veto, allow votes, and quota rules	6
2.4	Utility, cost, and constraints	6
3	The constructive theory	7
3.1	Coverage is submodular	7
3.2	Greedy residual coverage	7
3.3	Finite-validation empirical greedy	8
3.4	Benign overblocking as a conservative budget	10
3.5	Cost-aware and budgeted variants	11
3.6	Quota rules reduce aggregation search	11
3.7	Weighted quota as the first calibrated extension	12
3.8	Sparse compression of large committees	13
3.9	Paired evaluation and repeat allocation	14

3.10	Finite-candidate certification after structural search	15
3.11	Adaptive racing	15
4	The CASS-Greedy-Quota algorithm	16
4.1	Inputs	16
4.2	Offline construction procedure	17
4.3	Outputs	17
5	Implementation details	18
5.1	Data schema	18
5.2	Efficient residual coverage computation	18
5.3	Context recipe discipline	19
5.4	Choosing the benign-overblock budget	19
5.5	Quota scanning	19
5.6	Weighted quota implementation	20
5.7	Certifying against a baseline	20
6	Scaling and production deployment	20
6.1	Reference architecture	20
6.2	Parallel, serial, and staged serving	21
6.3	Timeouts and fallbacks	21
6.4	Logging for audit and learning	21
6.5	Monitoring metrics	22
6.6	Shadow deployment and canary release	22
6.7	Model stack cards	23
6.8	Re-evaluation triggers	23
7	Scaling the offline search	23
7.1	From hundreds to thousands of atoms	23
7.2	Pre-filtering rules	24
7.3	Cell-aware selection	24
7.4	Streaming updates	24
8	Validation design	25
8.1	Recommended data split	25
8.2	What to report	25
8.3	When not to deploy the stack	25
9	Extensions and limitations	26
9.1	Ordinary task performance	26
9.2	Common-mode failures	26
9.3	Richer interaction protocols	26
9.4	Label quality	26
9.5	Distribution shift	26
10	Minimal implementation blueprint	26
10.1	Prototype implementation	26
10.2	Core greedy pseudocode	27
10.3	Quota scan pseudocode	28

11 Conclusion

28

1 Problem statement

A practitioner has a task and a catalog of small or inexpensive models. The task may be a safety task, such as deciding whether to allow or block a user request, or an ordinary performance task, such as solving domain-specific questions. The practitioner does not want to deploy one large expensive model if a small committee can reach comparable performance at lower cost. The practitioner also has limited task-specific benchmark data. This creates a difficult selection problem:

Which small models should be composed, what simple context should they receive, how should their outputs be combined, and how much evidence is needed before deploying the resulting stack?

The challenge is that the design space is combinatorial. If there are p models and r context recipes, there are $p(r + 1)$ model-context pairs. If the system can use any subset of size at most k and any Boolean aggregation rule over k binary votes, then the number of candidate stacks is roughly

$$\sum_{s=1}^k \binom{p(r+1)}{s} 2^{2^s}.$$

Even for small k , this is too large to search directly. Worse, a large search space causes statistical over-selection: the best observed stack on a small validation set may not be the best stack in deployment.

This report develops a more constructive approach. Instead of searching arbitrary stacks, we restrict attention to families with classical mathematical structure:

Source of theory	Structure used	Practical consequence
Combinatorics / submodular optimization	Diminishing returns coverage	Greedily add complementary models
Voting theory	Monotone anonymous binary aggregation	Search quota thresholds, not all rules
Statistical decision theory	Weighted likelihood aggregation	Use weighted quotas when reliability is estimable
Concentration inequalities	Sparse approximation	Compress large committees into small committees
Bandit optimization	Successive elimination	Spend validation data on near-ties
Learning theory	Uniform validation bounds	Penalize search and certify on holdout

The resulting method is called **CASS-Greedy-Quota**. Here, CASS means *composition-aware stack selection*: the evaluated object is not just a model, but a complete stack design with model choices, context recipes, aggregation rules, and costs.

1.1 What this report is not claiming

The report deliberately avoids several over-strong claims.

1. It does not claim that greedy maximizes arbitrary net utility. Greedy is justified for coverage-style objectives under explicit constraints. Full net utility can be a difference of submodular functions and is harder.

2. It does not claim that majority vote is always optimal. Majority is only one quota rule.
3. It does not claim that weighted quota is always optimal. Weighted quota has a clean derivation under conditional independence and calibrated error rates, but real model errors can be correlated.
4. It does not claim that validation performance alone is sufficient. The final stack should be certified on a fresh holdout or by an equivalent procedure that controls adaptive reuse.

The aim is narrower and more useful: give a cheap, interpretable, and statistically disciplined procedure for constructing small committees.

2 Basic setup

2.1 Episodes, cells, and labels

An *episode* is one task instance. In a guardrail setting, an episode may be a user request plus relevant conversation context. In a question-answering setting, it may be a problem and an answer key. We denote an episode by x .

A *cell* is a task-relevant stratum. Examples include benign support requests, adversarial social-engineering attempts, ambiguous policy requests, high-value customer requests, or low-risk informational requests. Let

$$c(x) \in \mathcal{C}_{\text{cell}}$$

be the cell of episode x . Cells matter because deployment priorities and error rates are rarely uniform.

For a safety task, we use two broad sides:

$$N = \text{benign}, \quad A = \text{harmful/adversarial}.$$

For ordinary task performance, the distinction may instead be correct versus incorrect, or pass versus fail. The theory below is stated mostly for guardrail-like block/allow tasks because that is where coverage is most natural, but the implementation ideas extend to other binary task outcomes.

2.2 Model-context atoms

Let

$$M = \{m_1, \dots, m_p\}$$

be the catalog of candidate models. Let

$$H = \{h_0, h_1, \dots, h_r\}$$

be a deliberately small set of context recipes. Examples are:

$$\begin{aligned} h_0 &= \text{raw prompt only}, & h_1 &= \text{short task instruction}, \\ h_2 &= \text{short policy snippet}, & h_3 &= \text{retrieved top-}K \text{ snippet}. \end{aligned}$$

An *atom* is a model-context pair:

$$a = (m, h) \in \mathcal{A} = M \times H.$$

The key simplification is that context is treated as a small finite design choice. We do not attempt to model the internal semantics of context. We simply evaluate atoms.

For a guardrail task, atom a emits a binary block signal

$$b_a(x) \in \{0, 1\},$$

where $b_a(x) = 1$ means that atom a blocks or flags episode x . A stack is a subset

$$S \subseteq \mathcal{A}.$$

2.3 Serial veto, allow votes, and quota rules

A serial-veto stack blocks if any selected atom blocks:

$$B_S(x) = \max_{a \in S} b_a(x).$$

Equivalently, if $Q_a(x) = 1 - b_a(x)$ is an allow vote, then serial veto allows only if all selected atoms allow:

$$\text{allow} \iff \sum_{a \in S} Q_a(x) = |S|.$$

A quota rule generalizes this:

$$\text{allow} \iff \sum_{a \in S} Q_a(x) \geq q.$$

The special cases are:

Quota	Rule
$q = 1$	permissive OR: allow if any atom allows
$q = \lceil S /2 \rceil$	majority vote
$q = S $	unanimous allow / serial veto

2.4 Utility, cost, and constraints

For harmful episodes, blocking is good. For benign episodes, blocking is bad. Let P_A and P_N be the harmful and benign deployment distributions. Define harmful coverage

$$F(S) = \mathbb{E}_{x \sim P_A} [w_A(x) B_S(x)],$$

and benign blocking loss

$$L(S) = \mathbb{E}_{x \sim P_N} [w_N(x) B_S(x)],$$

where $w_A, w_N \geq 0$ are optional importance weights. The direct net utility of a serial-veto stack might be written

$$U_\lambda(S) = \lambda F(S) - L(S) - \text{Cost}(S),$$

where λ is the relative cost of harmful leakage. However, the difference $\lambda F(S) - L(S)$ is not generally submodular. Therefore the core construction uses the constrained problem

$$\max_{S \subseteq \mathcal{A}} F(S)$$

subject to

$$|S| \leq k, \quad L(S) \leq \beta, \quad \text{Cost}(S) \leq B.$$

This says: maximize harmful coverage subject to a deployed-model budget, a benign-overblock budget, and a cost or latency budget.

3 The constructive theory

3.1 Coverage is submodular

Submodularity is the mathematical form of diminishing returns. An atom is valuable if it catches harmful examples that the current stack misses. Once those examples are already caught, the same atom becomes less valuable.

Definition 3.1 (Monotone submodular set function). *A set function $F : 2^{\mathcal{A}} \rightarrow \mathbb{R}$ is monotone if $S \subseteq T$ implies $F(S) \leq F(T)$. It is submodular if for every $S \subseteq T \subseteq \mathcal{A}$ and $a \notin T$,*

$$F(S \cup \{a\}) - F(S) \geq F(T \cup \{a\}) - F(T).$$

Theorem 3.2 (Serial-veto harmful coverage is monotone submodular). *Let $b_a(x) \in \{0, 1\}$ for every atom $a \in \mathcal{A}$, and define*

$$B_S(x) = \max_{a \in S} b_a(x), \quad F(S) = \mathbb{E}_{x \sim P_A} [w(x) B_S(x)],$$

with $w(x) \geq 0$. Then F is monotone submodular.

Proof. If $S \subseteq T$, then $B_S(x) \leq B_T(x)$ for every x . Multiplying by $w(x) \geq 0$ and taking expectations gives $F(S) \leq F(T)$, so F is monotone.

For submodularity, define the marginal gain

$$\Delta(a \mid S) = F(S \cup \{a\}) - F(S).$$

For a fixed episode x ,

$$B_{S \cup \{a\}}(x) - B_S(x) = \mathbf{1}\{B_S(x) = 0, b_a(x) = 1\}.$$

Thus

$$\Delta(a \mid S) = \mathbb{E}_{x \sim P_A} [w(x) \mathbf{1}\{B_S(x) = 0, b_a(x) = 1\}].$$

If $S \subseteq T$, then $B_T(x) \geq B_S(x)$, so

$$\{B_T(x) = 0\} \subseteq \{B_S(x) = 0\}.$$

Therefore

$$\mathbf{1}\{B_T(x) = 0, b_a(x) = 1\} \leq \mathbf{1}\{B_S(x) = 0, b_a(x) = 1\}.$$

Multiplying by $w(x)$ and taking expectations gives $\Delta(a \mid T) \leq \Delta(a \mid S)$. Hence F is submodular. $\square$

3.2 Greedy residual coverage

The greedy algorithm starts with no atoms and repeatedly adds the atom with the largest residual harmful coverage.

Greedy residual coverage.

1. Set $S_0 = \emptyset$.
2. For $t = 1, \dots, k$, choose

$$a_t \in \arg \max_{a \notin S_{t-1}} [F(S_{t-1} \cup \{a\}) - F(S_{t-1})].$$

3. Set $S_t = S_{t-1} \cup \{a_t\}$.

Theorem 3.3 (Greedy coverage guarantee). *Let*

$$S^* \in \arg \max_{|S| \leq k} F(S)$$

be the best k -atom stack for harmful coverage. If F is nonnegative, monotone, and submodular with $F(\emptyset) = 0$, then the greedy stack S_k satisfies

$$F(S_k) \geq \left(1 - \frac{1}{e}\right) F(S^*).$$

Proof. Let $G_t = F(S^*) - F(S_t)$ be the gap after t greedy steps. By monotonicity,

$$F(S^*) \leq F(S_t \cup S^*),$$

so

$$G_t \leq F(S_t \cup S^*) - F(S_t).$$

By submodularity,

$$F(S_t \cup S^*) - F(S_t) \leq \sum_{a \in S^*} [F(S_t \cup \{a\}) - F(S_t)].$$

Because $|S^*| \leq k$, at least one atom in S^* has marginal gain at least G_t/k . Greedy chooses the best available atom, hence

$$F(S_{t+1}) - F(S_t) \geq \frac{G_t}{k}.$$

Therefore

$$G_{t+1} \leq \left(1 - \frac{1}{k}\right) G_t.$$

Iterating for k steps gives

$$G_k \leq \left(1 - \frac{1}{k}\right)^k F(S^*) \leq e^{-1} F(S^*).$$

Thus

$$F(S_k) = F(S^*) - G_k \geq (1 - e^{-1}) F(S^*).$$

□

This is the classic guarantee for monotone submodular maximization under a cardinality constraint, originating in the work of Nemhauser, Wolsey, and Fisher [12]. The practical implication is substantial: instead of enumerating $\binom{|\mathcal{A}|}{k}$ stacks, one obtains a constant-factor approximation with $O(k|\mathcal{A}|)$ marginal-gain searches.

3.3 Finite-validation empirical greedy

In deployment, F is unknown. We estimate it from harmful validation examples

$$x_1, \dots, x_{n_A} \sim P_A.$$

Assume $w(x)B_S(x) \in [0, 1]$ and define

$$\hat{F}(S) = \frac{1}{n_A} \sum_{i=1}^{n_A} w(x_i)B_S(x_i).$$

Empirical greedy uses $\hat{F}$ instead of F .

Theorem 3.4 (Empirical greedy under marginal estimation error). *Let $\widehat{S}_k$ be the stack produced by greedy using empirical marginals. Suppose that for every $|S| < k$ and every $a \notin S$,*

$$|\widehat{\Delta}(a | S) - \Delta(a | S)| \leq \eta,$$

where $\Delta(a | S) = F(S \cup \{a\}) - F(S)$ and $\widehat{\Delta}(a | S) = \widehat{F}(S \cup \{a\}) - \widehat{F}(S)$. Then

$$F(\widehat{S}_k) \geq \left(1 - \frac{1}{e}\right) F(S^*) - 2k\eta.$$

Proof. Let $G_t = F(S^*) - F(\widehat{S}_t)$. By the same submodularity argument as in the exact greedy proof, there exists some $a \in S^*$ whose true marginal after $\widehat{S}_t$ is at least G_t/k . Let a_t be the atom selected by empirical greedy. Since empirical marginals are within η of true marginals,

$$\widehat{\Delta}(a_t | \widehat{S}_t) \geq \widehat{\Delta}(a | \widehat{S}_t)$$

implies

$$\Delta(a_t | \widehat{S}_t) \geq \Delta(a | \widehat{S}_t) - 2\eta \geq \frac{G_t}{k} - 2\eta.$$

Therefore

$$G_{t+1} \leq \left(1 - \frac{1}{k}\right) G_t + 2\eta.$$

Iterating this recurrence for k steps gives

$$G_k \leq e^{-1} F(S^*) + 2k\eta,$$

and hence

$$F(\widehat{S}_k) \geq (1 - e^{-1}) F(S^*) - 2k\eta.$$

□

Corollary 3.5 (A simple uniform marginal bound). *Let N_{marg} be the number of marginals over which the empirical greedy analysis must be uniform. A crude bound is*

$$N_{\text{marg}} \leq \sum_{s=0}^{k-1} \binom{|\mathcal{A}|}{s} (|\mathcal{A}| - s) \leq k|\mathcal{A}|^k.$$

By Hoeffding's inequality and a union bound, with probability at least $1 - \delta$,

$$\eta = \sqrt{\frac{\log(2N_{\text{marg}}/\delta)}{2n_A}}$$

satisfies the assumption of the empirical greedy theorem.

Proof. For a fixed marginal, the empirical marginal is an average of bounded random variables in $[-1, 1]$ or, by expanding as a newly covered mass, in $[0, 1]$. Hoeffding's inequality bounds the probability that it differs from its expectation by more than η . Applying a union bound over N_{marg} candidate marginals and solving for η gives the stated expression. □

The point is not the exact constant. The point is that empirical greedy pays roughly for $k \log |\mathcal{A}|$ scale rather than the full exponential stack class. Hoeffding's inequality is the standard bounded-variable concentration tool used here [7].

3.4 Benign overblocking as a conservative budget

The coverage theorem rewards blocking harmful examples. But a guardrail that blocks everything would have perfect harmful coverage and terrible benign utility. We therefore need a benign-overblock constraint.

For benign examples, define

$$L(S) = \mathbb{E}_{x \sim P_N} [w_N(x) B_S(x)].$$

For each atom, define singleton benign loss

$$\ell_a = \mathbb{E}_{x \sim P_N} [w_N(x) b_a(x)].$$

Lemma 3.6 (Union-bound benign-loss control). *For every $S \subseteq \mathcal{A}$,*

$$L(S) \leq \sum_{a \in S} \ell_a.$$

Proof. For every x ,

$$B_S(x) = \mathbf{1}\{\exists a \in S : b_a(x) = 1\} \leq \sum_{a \in S} b_a(x).$$

Multiplying by $w_N(x) \geq 0$ and taking expectations gives

$$L(S) \leq \sum_{a \in S} \mathbb{E}[w_N(x) b_a(x)] = \sum_{a \in S} \ell_a.$$

□

Now estimate ℓ_a on n_N benign validation examples:

$$\hat{\ell}_a = \frac{1}{n_N} \sum_{i=1}^{n_N} w_N(x_i) b_a(x_i), \quad w_N(x_i) b_a(x_i) \in [0, 1].$$

Define

$$\rho = \sqrt{\frac{\log(2|\mathcal{A}|/\delta)}{2n_N}}.$$

Theorem 3.7 (High-probability benign-budget guarantee). *With probability at least $1 - \delta$, all atoms satisfy*

$$\ell_a \leq \hat{\ell}_a + \rho.$$

Consequently, any selected stack S satisfying

$$\sum_{a \in S} (\hat{\ell}_a + \rho) \leq \beta$$

also satisfies

$$L(S) \leq \beta.$$

Proof. For a fixed atom a , Hoeffding's inequality gives

$$\mathbb{P}(\ell_a > \hat{\ell}_a + \rho) \leq e^{-2n_N \rho^2}.$$

With the chosen ρ , this probability is at most $\delta/(2|\mathcal{A}|)$ up to the usual one-sided constant convention. A union bound over atoms gives simultaneous validity. Combining this with the previous lemma yields

$$L(S) \leq \sum_{a \in S} \ell_a \leq \sum_{a \in S} (\hat{\ell}_a + \rho) \leq \beta.$$

□

This is conservative because it ignores overlap among benign false blocks. However, it is easy to implement and auditable. In production, this is often more valuable than a sharper but opaque estimate.

3.5 Cost-aware and budgeted variants

The cardinality theorem uses $|S| \leq k$. Production systems usually care about cost and latency:

$$\text{Cost}(S) = \sum_{a \in S} c_a \leq B.$$

The corresponding mathematical problem is a budgeted maximum-coverage or submodular-knapsack problem. Budgeted maximum coverage and monotone submodular maximization under knapsack constraints have known constant-factor approximation algorithms, including $(1 - 1/e)$ -type results under standard formulations [9, 14]. A simple ratio-greedy heuristic chooses

$$a_t \in \arg \max_{a \notin S_{t-1}} \frac{\hat{\Delta}_A(a \mid S_{t-1})}{c_a},$$

subject to the benign and cost budgets. The report treats this ratio rule as the practical default, while noting that exact approximation constants require the more careful algorithms in the budgeted-coverage literature.

3.6 Quota rules reduce aggregation search

After selecting atoms, a designer still must aggregate their outputs. Arbitrary Boolean aggregation over k binary votes has 2^{2^k} possible rules. Voting theory gives a useful reduction.

Let $Q_i \in \{0, 1\}$ be the allow vote of atom i , where 1 means allow. An aggregation rule is a function

$$g : \{0, 1\}^k \rightarrow \{0, 1\},$$

where $g(Q) = 1$ means the final stack allows.

Definition 3.8 (Anonymous and monotone aggregation). *The rule g is anonymous if it depends only on the number of allow votes, $\sum_i Q_i$. It is monotone if changing any vote from block to allow cannot change the final decision from allow to block. Formally, if $Q_i \leq Q'_i$ for all i , then $g(Q) \leq g(Q')$.*

Theorem 3.9 (Anonymous monotone binary aggregation is a quota rule). *If $g : \{0, 1\}^k \rightarrow \{0, 1\}$ is anonymous and monotone, then there exists a quota $q \in \{0, 1, \dots, k + 1\}$ such that*

$$g(Q) = 1 \iff \sum_{i=1}^k Q_i \geq q.$$

Proof. Because g is anonymous, there is a function

$$\phi : \{0, 1, \dots, k\} \rightarrow \{0, 1\}$$

such that $g(Q) = \phi(\sum_i Q_i)$. Because g is monotone, ϕ is nondecreasing. Every nondecreasing binary-valued function on a finite ordered set is a threshold function. Let

$$q = \min\{j : \phi(j) = 1\},$$

with $q = k + 1$ if the set is empty and $q = 0$ if $\phi(j) = 1$ for all j . Then $\phi(j) = 1$ if and only if $j \geq q$, which proves the claim. $\square$

The theorem is simple, but its operational consequence is large. If anonymity and monotonicity are acceptable design constraints, the rule search collapses from 2^{2^k} Boolean functions to $k + 2$ thresholds. Majority-rule axiomatizations go back at least to May's theorem [10]; the present result is a simpler threshold characterization for binary anonymous monotone rules.

3.7 Weighted quota as the first calibrated extension

Anonymous quota treats all atoms equally. If some atoms are more reliable than others, weighted quota is the natural next step:

$$g(Q) = 1 \iff \sum_{i=1}^k \alpha_i Q_i \geq \tau.$$

There is a clean derivation under conditional independence.

Theorem 3.10 (Likelihood-ratio derivation of weighted quota). *Suppose the true episode type is either benign N or harmful A . For atom i , let*

$$t_i = \mathbb{P}(Q_i = 1 \mid N), \quad f_i = \mathbb{P}(Q_i = 1 \mid A),$$

where $Q_i = 1$ means allow. Assume conditional independence of the votes given the type. Then the Bayes decision rule that allows when the posterior evidence for N over A exceeds a threshold has weighted-quota form

$$\sum_i \alpha_i Q_i \geq \tau,$$

where

$$\alpha_i = \log \frac{t_i(1 - f_i)}{f_i(1 - t_i)}.$$

Proof. The likelihood ratio is

$$\frac{\mathbb{P}(Q \mid N)}{\mathbb{P}(Q \mid A)} = \prod_{i=1}^k \frac{\mathbb{P}(Q_i \mid N)}{\mathbb{P}(Q_i \mid A)}.$$

Taking logs gives

$$\log \frac{\mathbb{P}(Q | N)}{\mathbb{P}(Q | A)} = \sum_i \left[Q_i \log \frac{t_i}{f_i} + (1 - Q_i) \log \frac{1 - t_i}{1 - f_i} \right].$$

Rearranging terms,

$$\log \frac{\mathbb{P}(Q | N)}{\mathbb{P}(Q | A)} = C + \sum_i Q_i \log \frac{t_i(1 - f_i)}{f_i(1 - t_i)},$$

where $C = \sum_i \log((1 - t_i)/(1 - f_i))$ does not depend on Q . Comparing this log-likelihood ratio to the Bayes threshold yields a weighted threshold rule with the stated weights. $\square$

This connects small-model committee aggregation to classical noisy-observer models such as Dawid and Skene’s error-rate model [3]. However, the theorem requires assumptions that may fail for language models, especially conditional independence. In practice, weighted quota should be cross-fit and certified on held-out data.

3.8 Sparse compression of large committees

A large weighted committee may be useful as an offline design object, but too expensive to deploy. Concentration gives a simple compression principle.

Suppose there are M atoms. On validation example i , atom m emits a bounded score $v_{im} \in [0, 1]$. A large weighted committee has weights $w \in \Delta^M$ and score

$$s_i = \sum_{m=1}^M w_m v_{im}.$$

Sample L atoms independently from distribution w :

$$m_1, \dots, m_L \sim w.$$

Define the sparse committee score

$$\tilde{s}_i = \frac{1}{L} \sum_{\ell=1}^L v_{i, m_\ell}.$$

Theorem 3.11 (Sparse committee approximation on a validation table). *With probability at least $1 - \delta$,*

$$\max_{1 \leq i \leq n} |\tilde{s}_i - s_i| \leq \epsilon$$

provided

$$L \geq \frac{\log(2n/\delta)}{2\epsilon^2}.$$

Proof. For fixed i , the random variables $v_{i, m_1}, \dots, v_{i, m_L}$ are independent, bounded in $[0, 1]$, and have mean s_i . Hoeffding’s inequality gives

$$\mathbb{P}(|\tilde{s}_i - s_i| > \epsilon) \leq 2e^{-2L\epsilon^2}.$$

A union bound over the n validation examples gives

$$\mathbb{P}\left(\max_i |\tilde{s}_i - s_i| > \epsilon\right) \leq 2ne^{-2L\epsilon^2}.$$

The stated lower bound on L makes this probability at most δ . $\square$

Corollary 3.12 (Margin preservation). *Suppose the large committee uses threshold τ and decides allow when $s_i \geq \tau$. If $|s_i - \tau| > \epsilon$, then the sparse committee makes the same decision on example i whenever $|\tilde{s}_i - s_i| \leq \epsilon$.*

The sparse theorem is a validation-table statement, not a universal deployment guarantee. After compression, the sparse stack still needs holdout certification.

3.9 Paired evaluation and repeat allocation

The methods above require evaluating candidate stacks. Paired evaluation is usually more efficient than unpaired evaluation because the same examples are used for all candidates.

Let $Z_a(x)$ be the bounded utility of stack a on example x . For two stacks a and b , the paired difference is

$$D(x) = Z_a(x) - Z_b(x).$$

The paired estimator is

$$\hat{\Delta}_{a,b} = \frac{1}{n} \sum_{i=1}^n D(x_i).$$

Proposition 3.13 (Paired comparison variance). *If Z_a and Z_b are evaluated on the same examples, then*

$$\text{Var}(\hat{\Delta}_{a,b}) = \frac{1}{n} [\text{Var}(Z_a) + \text{Var}(Z_b) - 2 \text{Cov}(Z_a, Z_b)].$$

If the two stacks are evaluated on independent examples of equal size, the covariance term is absent. Therefore paired evaluation is more efficient whenever the same-example covariance is positive.

Proof. The variance of $D = Z_a - Z_b$ is

$$\text{Var}(D) = \text{Var}(Z_a) + \text{Var}(Z_b) - 2 \text{Cov}(Z_a, Z_b).$$

Averaging n independent paired differences divides the variance by n . Independent unpaired means have zero covariance, giving the comparison. $\square$

If model outputs are stochastic, repeated runs can be useful. For a fixed stack a , let

$$p_a(x) = \mathbb{E}_\xi[Z_a(x, \xi) \mid x].$$

Define between-example variance

$$\tau_a^2 = \text{Var}_x(p_a(x))$$

and within-example stochastic variance

$$\nu_a^2 = \mathbb{E}_x[\text{Var}_\xi(Z_a(x, \xi) \mid x)].$$

If we evaluate n examples and r repeated runs per example, then the repeated-run estimator has variance

$$\frac{\tau_a^2}{n} + \frac{\nu_a^2}{nr}.$$

Thus repeated runs reduce only the within-example component. If new examples cost c_E and each run costs c_R , the approximate variance-minimizing repeat count is

$$r^* \approx \sqrt{\frac{c_E}{c_R} \cdot \frac{\nu_a^2}{\tau_a^2}}.$$

This result follows directly from the law of total variance and a one-variable optimization.

3.10 Finite-candidate certification after structural search

After greedy construction, quota scanning, optional weighting, and optional compression, the final candidate set should be small. Let it be

$$\mathcal{C} = \{c_1, \dots, c_K\}.$$

The final certification should be performed on fresh data not used to choose $\mathcal{C}$, or on a carefully protected holdout.

Theorem 3.14 (Finite-candidate holdout bound). *Assume each candidate $c \in \mathcal{C}$ has bounded utility $Z_c \in [0, 1]$. Let*

$$U(c) = \mathbb{E}[Z_c], \quad \widehat{U}(c) = \frac{1}{n} \sum_{i=1}^n Z_c(x_i)$$

be its true and holdout empirical utilities. If the holdout examples are independent of the candidate-selection procedure, then with probability at least $1 - \delta$,

$$\sup_{c \in \mathcal{C}} |\widehat{U}(c) - U(c)| \leq \sqrt{\frac{\log(2K/\delta)}{2n}}.$$

Consequently, if $\widehat{c} \in \arg \max_{c \in \mathcal{C}} \widehat{U}(c)$ and $c^ \in \arg \max_{c \in \mathcal{C}} U(c)$, then*

$$U(c^*) - U(\widehat{c}) \leq 2\sqrt{\frac{\log(2K/\delta)}{2n}}.$$

Proof. For each fixed c , Hoeffding's inequality gives

$$\mathbb{P}(|\widehat{U}(c) - U(c)| > \epsilon) \leq 2e^{-2n\epsilon^2}.$$

A union bound over K candidates yields the uniform deviation bound. On the event that all deviations are at most ϵ ,

$$U(c^*) \leq \widehat{U}(c^*) + \epsilon \leq \widehat{U}(\widehat{c}) + \epsilon \leq U(\widehat{c}) + 2\epsilon.$$

Substituting $\epsilon = \sqrt{\log(2K/\delta)/(2n)}$ proves the theorem. $\square$

This theorem explains why structural search matters. A small post-search candidate set has a much smaller certification penalty than the original combinatorial space. Adaptive data analysis research warns that repeatedly reusing the same holdout can invalidate naive inference [4]; the cleanest practical response is to separate construction data from final certification data.

3.11 Adaptive racing

If several candidates remain, evaluate them adaptively rather than spending equal benchmark budget on obviously weak candidates.

Let each candidate j have true utility U_j and bounded samples in $[0, 1]$. Let $\widehat{U}_j(n)$ be the empirical mean after n samples for candidate j . Define the confidence radius

$$r_j(n) = \sqrt{\frac{\log(4Kn^2/\delta)}{2n}}.$$

A successive-elimination rule removes candidate j if

$$\widehat{U}_j(n_j) + r_j(n_j) < \max_{\ell} \left[\widehat{U}_{\ell}(n_{\ell}) - r_{\ell}(n_{\ell}) \right].$$

Theorem 3.15 (Soundness of confidence racing). *With probability at least $1 - \delta$, the best candidate is never eliminated by the above rule.*

Proof. For each candidate j and sample count n , Hoeffding’s inequality gives

$$\mathbb{P}(|\widehat{U}_j(n) - U_j| > r_j(n)) \leq 2e^{-2nr_j(n)^2} = \frac{\delta}{2Kn^2}.$$

A union bound over $j = 1, \dots, K$ and $n = 1, 2, \dots$ gives simultaneous confidence intervals with probability at least $1 - \delta \sum_{n \geq 1} (2n^2)^{-1}$, which is at least $1 - \delta$ up to harmless constants. On this event, suppose the best candidate j^* were eliminated. Then for some ℓ ,

$$\widehat{U}_{j^*} + r_{j^*} < \widehat{U}_\ell - r_\ell.$$

The confidence event implies

$$U_{j^*} \leq \widehat{U}_{j^*} + r_{j^*} < \widehat{U}_\ell - r_\ell \leq U_\ell,$$

contradicting optimality of j^* . □

This is a simple form of best-arm identification. More refined algorithms, such as successive rejects, are studied in the bandit literature [1, 8]. The operational lesson is to spend additional evaluation mostly on near-ties.

4 The CASS-Greedy-Quota algorithm

4.1 Inputs

The practitioner provides:

- a model catalog M ;
- a small context recipe set H ;
- a harmful/adversarial validation set $\mathcal{D}_A$;
- a benign validation set $\mathcal{D}_N$;
- optional cell labels and cell weights;
- a maximum deployed atom count k ;
- a benign-overblock budget β ;
- a model-call, dollar, or latency budget B ;
- a final certification holdout $\mathcal{D}_{\text{cert}}$.

The atoms are $\mathcal{A} = M \times H$.

4.2 Offline construction procedure

Algorithm: CASS-Greedy-Quota.

1. **Create atoms.** For every model $m \in M$ and context recipe $h \in H$, define atom $a = (m, h)$.
2. **Log outputs.** Run every atom on the paired validation examples. Store block signals $b_a(x)$, allow votes $Q_a(x) = 1 - b_a(x)$, scores if available, latency, cost, model version, prompt hash, and context-recipe ID.
3. **Estimate benign singleton loss.** For each atom, compute $\hat{\ell}_a$ and $\text{UCB}_a = \hat{\ell}_a + \rho$.
4. **Greedy residual harmful coverage.** Starting from $S_0 = \emptyset$, repeatedly add the feasible atom with largest residual harmful coverage per cost:

$$a_t \in \arg \max_a \frac{\hat{F}(S_{t-1} \cup \{a\}) - \hat{F}(S_{t-1})}{c_a}.$$

Only add atoms satisfying

$$\sum_{a \in S_{t-1} \cup \{a_t\}} \text{UCB}_a \leq \beta, \quad \sum_{a \in S_{t-1} \cup \{a_t\}} c_a \leq B.$$

Stop after k atoms or when no feasible atom has meaningful marginal gain.

5. **Create greedy prefixes.** Keep $S_1, S_2, \dots, S_k$, not just the final prefix.
6. **Scan quota rules.** For each prefix S_t , evaluate quota rules $q = 1, \dots, t$.
7. **Optionally fit weighted quota.** If the validation set is large enough, estimate atom weights by cross-fitting and scan thresholds. Otherwise skip this step.
8. **Optionally compress.** If a large weighted committee is useful offline, compress it to a small committee and add the compressed candidate to the shortlist.
9. **Race candidates.** Use confidence racing among the baseline, greedy prefixes, quota candidates, weighted candidates, and compressed candidates.
10. **Certify final candidates.** On a fresh paired holdout, report utility, uncertainty intervals, cost, latency, benign loss, harmful coverage, and comparison to the best single-model baseline.

4.3 Outputs

The method should not output only a winning stack. It should output a deployment decision record:

1. selected atoms and their model/context versions;
2. aggregation rule and threshold;
3. expected harmful coverage and benign overblock estimates;
4. cost and latency profile;
5. final holdout confidence interval versus the baseline;
6. unresolved near-ties, if any;
7. rollback candidate;
8. monitoring metrics and alert thresholds.

5 Implementation details

5.1 Data schema

The central engineering artifact is a table of atom outputs on paired examples. A minimal schema is:

Field	Meaning
<code>example_id</code>	Stable identifier for the validation episode
<code>cell_id</code>	Task/risk cell
<code>label_side</code>	Benign, harmful, or task class
<code>weight</code>	Optional cell/example importance weight
<code>atom_id</code>	Model-context atom identifier
<code>model_id, model_version</code>	Exact model and version
<code>context_recipe_id</code>	Context recipe identifier
<code>prompt_hash</code>	Hash of prompt/template/context recipe
<code>seed, temperature</code>	Runtime stochasticity controls, if applicable
<code>raw_output</code>	Raw model output, when storage policy allows
<code>parsed_vote</code>	Binary block or allow vote
<code>score</code>	Optional calibrated or uncalibrated score
<code>latency_ms, cost_units</code>	Runtime measurements
<code>parser_version</code>	Output parser version

This table allows most construction steps to be performed as deterministic table operations after model calls have been logged.

5.2 Efficient residual coverage computation

For binary block signals, store each atom’s harmful block vector as a bitset:

$$v_a \in \{0, 1\}^{n_A}.$$

Maintain the residual-uncovered harmful vector

$$r_S = \mathbf{1} - B_S.$$

The marginal gain of atom a is the weighted count of examples where both $r_S = 1$ and $v_a = 1$:

$$\hat{\Delta}_A(a \mid S) = \frac{1}{n_A} \sum_i w_i r_{S,i} v_{a,i}.$$

After adding atom a , update

$$r_{S \cup \{a\}} = r_S \wedge \neg v_a.$$

Bitset operations make this fast. With $n_A = 100,000$ harmful examples and 1,000 atoms, the raw binary matrix has 10^8 bits, about 12.5 MB before metadata and indexing overhead.

For large catalogs, use *lazy greedy*: keep an upper-bound priority queue of marginal gains, recompute the top candidate’s true current marginal, and accept it only if it remains on top. Submodularity ensures marginal gains only decrease, so old marginals are valid upper bounds.

5.3 Context recipe discipline

The context set H should be small. A useful starting set is:

1. raw input only;
2. raw input plus short task instruction;
3. raw input plus short policy excerpt;
4. raw input plus one retrieved snippet;
5. raw input plus compact conversation summary, if conversation state matters.

Do not treat every prompt tweak as a new context recipe unless there is enough validation data. Prompt and context search is still search; it consumes statistical degrees of freedom.

5.4 Choosing the benign-overblock budget

The benign budget β should be an operational policy choice, not just a tuning knob. Examples:

- maximum allowed false-block rate on ordinary benign requests;
- maximum allowed weighted false-block loss, where high-value benign requests carry larger weights;
- maximum allowed escalation rate for human review;
- cell-specific budgets, such as stricter limits for urgent support requests.

If budgets are cell-specific, apply the benign UCB constraint inside each cell:

$$\sum_{a \in S} (\hat{\ell}_{a,c} + \rho_c) \leq \beta_c.$$

5.5 Quota scanning

For each greedy prefix S_t , compute allow votes $Q_a(x) = 1 - b_a(x)$ and vote count

$$J_t(x) = \sum_{a \in S_t} Q_a(x).$$

The quota- q rule allows if $J_t(x) \geq q$. Since $J_t(x)$ is already computed, scanning all $q = 1, \dots, t$ is cheap.

For guardrails, high q is more conservative because it requires more atoms to allow. Low q is more permissive. The validation utility curve over q should be reported because it reveals the cost of stricter safety.

5.6 Weighted quota implementation

Weighted quota should be used only with care. Recommended implementation:

1. Split the selection data into folds.
2. On each training fold, estimate smoothed rates

$$\hat{t}_a = \mathbb{P}(Q_a = 1 \mid N), \quad \hat{f}_a = \mathbb{P}(Q_a = 1 \mid A).$$

3. Use smoothed weights

$$\hat{\alpha}_a = \log \frac{\hat{t}_a(1 - \hat{f}_a)}{\hat{f}_a(1 - \hat{t}_a)}.$$

4. Evaluate thresholds on the held-out fold.
5. Refit only after selecting the model class, then certify on final holdout.

Smoothing is essential because zero empirical false-pass rates can create infinite weights.

5.7 Certifying against a baseline

The final report should compare the selected stack to a baseline, usually the best single cheap model or current production system. On a paired holdout, define per-example utility difference

$$D_i = Z_{\text{stack}}(x_i) - Z_{\text{base}}(x_i).$$

Report

$$\hat{\Delta} = \frac{1}{n} \sum_i D_i$$

and a confidence interval using either empirical Bernstein, paired bootstrap, or a conservative Hoeffding interval. For bounded $D_i \in [-1, 1]$, Hoeffding gives

$$\mathbb{P}(|\hat{\Delta} - \Delta| > \epsilon) \leq 2e^{-n\epsilon^2/2}.$$

The deployment decision should state whether the lower confidence bound clears zero after accounting for cost.

6 Scaling and production deployment

6.1 Reference architecture

A production-like deployment has six components:

1. **Model registry.** Stores model versions, serving endpoints, costs, latency distributions, and deprecation status.
2. **Context registry.** Stores context recipe IDs, prompt templates, retrieval settings, policy snippets, and hashes.
3. **Atom runner.** Executes model-context atoms and returns parsed votes, scores, raw outputs when allowed, latency, and errors.

4. **Stack evaluator.** Applies the aggregation rule to atom outputs.
5. **Decision service.** Exposes the deployed stack to the application, with fallbacks and timeouts.
6. **Monitoring and evaluation loop.** Logs production outcomes, samples for labeling, detects drift, and triggers re-evaluation.

This separation reduces hidden coupling, a known source of technical debt in machine-learning systems [13]. Production-readiness checklists and monitoring rubrics are also standard recommendations for deployed ML systems [2].

6.2 Parallel, serial, and staged serving

There are three common serving modes.

Parallel committee. All atoms run at once, then the quota rule aggregates. This minimizes latency but maximizes concurrent model-call cost.

Serial veto. Atoms run in a fixed order and the system blocks as soon as one atom blocks. This can reduce average cost if blocking happens early, but increases worst-case latency unless calls are parallelized.

Staged triage. A cheap high-recall atom runs first. Easy cases are decided immediately; uncertain cases route to a larger committee or stronger model. This is often the best production compromise, but it should be evaluated as a distinct protocol because its output distribution differs from the parallel committee.

6.3 Timeouts and fallbacks

Every deployed atom should have a timeout policy. Examples:

- timeout counts as block in high-risk safety cells;
- timeout counts as abstain and triggers fallback in ordinary cells;
- timeout routes to a baseline model or human review;
- repeated atom failures remove the atom from service and activate rollback stack.

The timeout policy is part of the stack and must be included in evaluation.

6.4 Logging for audit and learning

Production logs should include:

- request ID and timestamp;
- cell prediction or routing label;
- selected stack version;
- atom versions and context recipe IDs;

- atom votes, scores, latency, cost, errors, and timeouts;
- final decision and aggregation threshold;
- downstream user or reviewer outcome, if later available;
- privacy-safe raw-output references or hashes.

For sensitive systems, raw text may not be stored; store hashes, redacted features, or approved audit samples instead.

6.5 Monitoring metrics

Monitor at least:

1. traffic mix by cell;
2. final allow/block/escalation rates by cell;
3. atom disagreement rates;
4. residual harmful misses found by review or red teaming;
5. benign overblock estimates from sampled audits;
6. cost and latency distributions;
7. atom timeout and parser-error rates;
8. drift in retrieved context sources;
9. quota margin distribution, e.g. $|\sum_i Q_i - q|$;
10. gap between shadow-stack and production-stack decisions.

Disagreement and small quota margins are useful sampling signals because they identify boundary cases for labeling.

6.6 Shadow deployment and canary release

A safe rollout path is:

1. **Offline validation.** Build and certify the candidate stack on historical and synthetic validation data.
2. **Shadow mode.** Run the stack on live traffic without affecting user-visible decisions. Compare against production decisions.
3. **Canary.** Route a small percentage of eligible traffic to the stack, with strict monitoring and rollback.
4. **Progressive rollout.** Increase traffic only if cell-level metrics remain inside guardrails.
5. **Post-deployment audit.** Sample decisions for labeling, especially disagreements and low-margin cases.

6.7 Model stack cards

Each deployed stack should have a short *stack card*, analogous in spirit to model cards [11]. It should document:

- intended task and excluded uses;
- selected atoms and versions;
- context recipes;
- aggregation rule;
- validation datasets and cells;
- harmful coverage and benign-overblock estimates;
- known failure modes;
- monitoring plan;
- rollback plan;
- date and owner.

6.8 Re-evaluation triggers

Re-run CASS-Greedy-Quota when:

- traffic mix changes materially;
- a model version changes;
- a context recipe or retrieval source changes;
- the policy changes;
- monitoring detects increased misses or overblocks;
- new cheaper models enter the catalog;
- latency or cost budgets change.

Continuous delivery and automation pipelines for ML emphasize repeatable data, model, and deployment workflows; the same principle applies here [6].

7 Scaling the offline search

7.1 From hundreds to thousands of atoms

For hundreds of atoms, a single validation table and bitset implementation is enough. For thousands or tens of thousands of atoms, use:

- distributed atom execution;
- columnar storage for outputs;

- bit-packed binary matrices;
- lazy greedy marginal recomputation;
- pre-filtering by singleton benign UCB and minimum harmful coverage;
- cell-specific indexing;
- approximate greedy on sampled harmful examples followed by exact evaluation of finalists.

7.2 Pre-filtering rules

Before greedy, remove atoms that are clearly dominated. Atom a is weakly dominated by atom b on the validation table if:

$$\begin{aligned} b_b(x) &\geq b_a(x) && \text{for all harmful examples,} \\ b_b(x) &\leq b_a(x) && \text{for all benign examples,} \\ c_b &\leq c_a. \end{aligned}$$

Such dominance will be rare with noisy models, but approximate dominance is useful for pruning. Also drop atoms that violate singleton constraints:

$$\widehat{\ell}_a + \rho > \beta$$

if no stack containing the atom can satisfy the benign budget.

7.3 Cell-aware selection

If there are multiple harmful cells, define

$$F(S) = \sum_{c \in A} w_c F_c(S).$$

Greedy remains valid because nonnegative weighted sums of submodular functions are submodular. For high-priority cells, either increase w_c or require minimum coverage constraints:

$$F_c(S) \geq \gamma_c.$$

Minimum coverage constraints make the optimization harder, but they are often necessary for safety-critical cells. A practical approach is to run greedy separately per critical cell to generate candidates, then merge and quota-scan candidates.

7.4 Streaming updates

When new labeled examples arrive, do not necessarily rebuild from scratch. Maintain:

- atom output columns for new examples;
- current residual coverage vectors;
- updated benign UCBs;
- drift metrics by cell.

If marginal rankings change substantially or constraints are violated, trigger a full rebuild and holdout certification.

8 Validation design

8.1 Recommended data split

Use at least three splits when possible:

1. **Construction split.** Used for greedy selection, quota scanning, and exploratory analysis.
2. **Racing split.** Used to eliminate weak candidates and narrow the shortlist.
3. **Certification split.** Used exactly once for final reporting and deployment decision.

If data are extremely scarce, use cross-fitting for construction but preserve a final untouched certification set. Reusing the same holdout repeatedly is a known source of adaptive overfitting [4].

8.2 What to report

A final evaluation report should include:

- performance by cell, not only pooled performance;
- paired comparison to baseline;
- confidence intervals;
- cost and latency intervals;
- benign-overblock UCB budget status;
- harmful coverage by cell;
- disagreement and margin distributions;
- examples of residual misses and false blocks;
- unresolved alternatives.

8.3 When not to deploy the stack

Do not deploy if:

1. the lower confidence bound versus baseline is not positive after cost;
2. the benign-overblock UCB exceeds the budget;
3. performance gains are concentrated in low-priority cells while losses occur in high-priority cells;
4. shadow deployment reveals drift from offline validation;
5. atom disagreement is high on cells where final decisions need high certainty;
6. the selected stack depends on a context source that is unstable or hard to monitor.

9 Extensions and limitations

9.1 Ordinary task performance

For non-safety tasks, define an atom’s success indicator

$$s_a(x) = \mathbf{1}\{\text{atom } a \text{ solves } x\}.$$

A committee that succeeds if any atom solves the example has coverage

$$F(S) = \mathbb{E}[\max_{a \in S} s_a(x)],$$

which is again monotone submodular. This supports greedy selection of complementary solvers. Aggregation is more complex when final answers must be synthesized, but the same principle applies to selecting a candidate-answer generator set before using a small verifier or voting rule.

9.2 Common-mode failures

Coverage methods help only when atoms have complementary errors. If all atoms miss the same harmful examples, residual coverage will quickly plateau. This is not a failure of the method; it is useful diagnostic information. A flat greedy marginal curve means the catalog lacks diversity for the task.

9.3 Richer interaction protocols

Debate, sequential critique, and rationale sharing can be useful, but they are not part of the core theorem. Treat each such design as a new protocol candidate and evaluate it directly. Do not assume that interaction improves correctness; it may also induce herding or correlated errors.

9.4 Label quality

All guarantees assume that validation labels and cell assignments are meaningful. If labels are noisy, the method optimizes noisy targets. For high-stakes applications, measure inter-annotator agreement, use adjudication, and track label uncertainty.

9.5 Distribution shift

The holdout certificate is about the distribution represented by the holdout. Production drift can invalidate it. This is why monitoring, shadow runs, and re-evaluation triggers are part of the method rather than optional engineering details.

10 Minimal implementation blueprint

10.1 Prototype implementation

A minimal Python implementation needs:

1. a dataframe of examples with labels, cells, and weights;
2. an atom registry with model endpoint and context recipe;
3. an execution loop that writes atom outputs to a columnar table;

4. a greedy selection module;
5. a quota scanner;
6. a holdout evaluator;
7. a stack-card generator.

10.2 Core greedy pseudocode

Input:

```
harmful_matrix B_A[n_A, num_atoms]
benign_singleton_ucb ucb[num_atoms]
atom_cost cost[num_atoms]
max_atoms k
benign_budget beta
cost_budget B
```

Initialize:

```
selected = []
residual = ones(n_A)
benign_spent = 0
cost_spent = 0
```

For t in 1..k:

```
best_atom = None
best_score = -inf
```

For each atom a not in selected:

```
if benign_spent + ucb[a] > beta: continue
if cost_spent + cost[a] > B: continue
```

```
marginal = weighted_sum(residual AND B_A[:, a])
score = marginal / cost[a]
```

```
if score > best_score:
    best_score = score
    best_atom = a
```

```
if best_atom is None or best_score <= min_gain:
    break
```

```
selected.append(best_atom)
residual = residual AND NOT B_A[:, best_atom]
benign_spent += ucb[best_atom]
cost_spent += cost[best_atom]
```

Return all prefixes of selected

10.3 Quota scan pseudocode

Input:

selected prefixes $S_1, \dots, S_k$
allow_votes $Q[n_examples, num_atoms]$
labels and utility function u

```

candidates = []
For each prefix  $S_t$ :
    counts = row_sum( $Q[:, S_t]$ )
    For  $q$  in  $1..t$ :
        decision = (counts  $\geq q$ )
        utility = mean( $u(\text{decision}, \text{labels})$ ) - cost( $S_t$ )
        candidates.append(( $S_t, q, \text{utility}$ ))

```

Return candidates sorted by utility

11 Conclusion

The main contribution of CASS-Greedy-Quota is not a new universal ensemble rule. It is a disciplined way to make small-model composition searchable, interpretable, and certifiable under limited task-specific data.

The method rests on four practical reductions:

1. model-context pairs become finite atoms;
2. serial-veto harmful coverage is submodular, so greedy residual selection gives a constant-factor approximation to the best k -atom coverage stack;
3. monotone anonymous aggregation reduces to quota search;
4. large weighted committees can be compressed and candidate stacks can be evaluated by racing and final holdout certification.

The resulting practitioner instruction is simple:

Build a paired atom-output table. Greedily cover residual failures under benign and cost budgets. Tune a quota. Race the finalists. Certify on a fresh holdout. Deploy with monitoring and rollback.

This is the right level of theory for the deployment problem: strong enough to reduce an exponential search space, but modest enough to implement and audit in production.

References

- [1] Jean-Yves Audibert, Sébastien Bubeck, and Rémi Munos. Best arm identification in multi-armed bandits. In *Proceedings of COLT*, 2010. <https://www.learningtheory.org/colt2010/papers/59Audibert.pdf>
- [2] Eric Breck, Shanqing Cai, Eric Nielsen, Michael Salib, and D. Sculley. The ML test score: A rubric for ML production readiness and technical debt reduction. In *IEEE International Conference on Big Data*, 2017. <https://research.google.com/pubs/archive/aad9f93b86b7addfea4c419b9100c6cdd26cacea.pdf>
- [3] A. Philip Dawid and Allan M. Skene. Maximum likelihood estimation of observer error-rates using the EM algorithm. *Journal of the Royal Statistical Society, Series C*, 28(1):20–28, 1979. DOI: <https://doi.org/10.2307/2346806>
- [4] Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth. Generalization in adaptive data analysis and holdout reuse. *Advances in Neural Information Processing Systems*, 2015. <https://arxiv.org/abs/1506.02629>
- [5] Uriel Feige. A threshold of $\ln n$ for approximating set cover. *Journal of the ACM*, 45(4):634–652, 1998. DOI: <https://doi.org/10.1145/285055.285059>
- [6] Google Cloud Architecture Center. MLOps: Continuous delivery and automation pipelines in machine learning. Last reviewed August 28, 2024. <https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>
- [7] Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963. DOI: <https://doi.org/10.1080/01621459.1963.10500830>
- [8] Emilie Kaufmann, Olivier Cappé, and Aurélien Garivier. On the complexity of best-arm identification in multi-armed bandit models. *Journal of Machine Learning Research*, 17(1):1–42, 2016. <https://jmlr.org/papers/v17/kaufman16a.html>
- [9] Samir Khuller, Anna Moss, and Joseph Naor. The budgeted maximum coverage problem. *Information Processing Letters*, 70(1):39–45, 1999. DOI: [https://doi.org/10.1016/S0020-0190\(99\)00031-9](https://doi.org/10.1016/S0020-0190(99)00031-9)
- [10] Kenneth O. May. A set of independent necessary and sufficient conditions for simple majority decision. *Econometrica*, 20(4):680–684, 1952. DOI: <https://doi.org/10.2307/1907651>
- [11] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, 2019. <https://arxiv.org/abs/1810.03993>
- [12] George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. An analysis of approximations for maximizing submodular set functions—I. *Mathematical Programming*, 14:265–294, 1978. DOI: <https://doi.org/10.1007/BF01588971>
- [13] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. Hidden technical debt in

- machine learning systems. In *Advances in Neural Information Processing Systems*, 2015. <https://papers.nips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems>
- [14] Maxim Sviridenko. A note on maximizing a submodular set function subject to a knapsack constraint. *Operations Research Letters*, 32(1):41–43, 2004. DOI: [https://doi.org/10.1016/S0167-6377\(03\)00062-2](https://doi.org/10.1016/S0167-6377(03)00062-2)
- [15] H. Peyton Young. Condorcet’s theory of voting. *American Political Science Review*, 82(4):1231–1244, 1988. DOI: <https://doi.org/10.2307/1961757>